

COMP30026

Models of Computation

Cezary Kaliszyk and **William Umboh**

Lecture 17 – Fooling Sets (Part 2), Nonregularity via Closure
Properties

Caution

- Slides are designed as presentation aid to go along with lecture, not as study material
- Slides not designed to make sense on their own

Announcements

- Friday is public holiday: no tutorials
- Next week is no teaching period: no tutorials
- Office hours:
 - Cezary: Thursdays at 10 am till end of semester (Lectures 1 – 15)
 - William: Thursdays at 3 pm in Melbourne Connect Open Area 2209 (Lectures 16 onwards) Free coffee!
- Make use of student hub!
- Assignment 2
 - Released Monday W10, due Monday W12
 - Solutions and feedback released Monday W13

Announcements

- Rose will be leading A1 consultation
 - Thursday 24 Sep 9:30 am - 11:30 am
 - Melbourne Connect Rm 4207

Today

- Review fooling sets
 - Examples and Intuition
- Proving nonregularity via closure properties

Recap: What it means to prove nonregularity

Recall

1. Language is *regular* if and only if some DFA recognizes it
2. DFA M recognizes language L :

For every string x , M accepts x if and only if $x \in L$

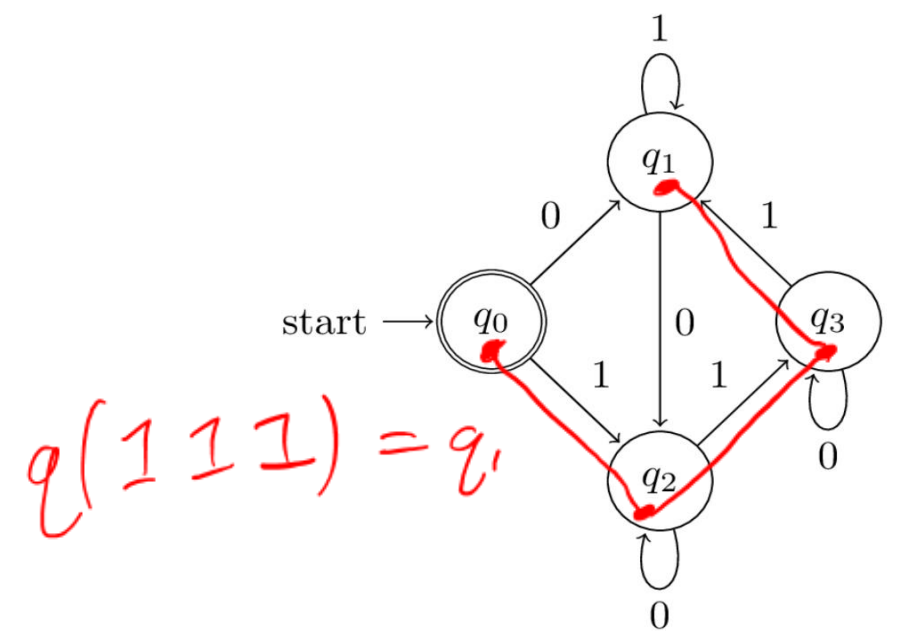
To show L is *not regular*, need to show no DFA recognizes it, i.e.

For **every** DFA M , there exists a string x on which M makes a mistake:

- M accepts x but $x \notin L$; or
- M rejects x but $x \in L$

What does it mean for a DFA to accept a string?

Recap: Computation Paths

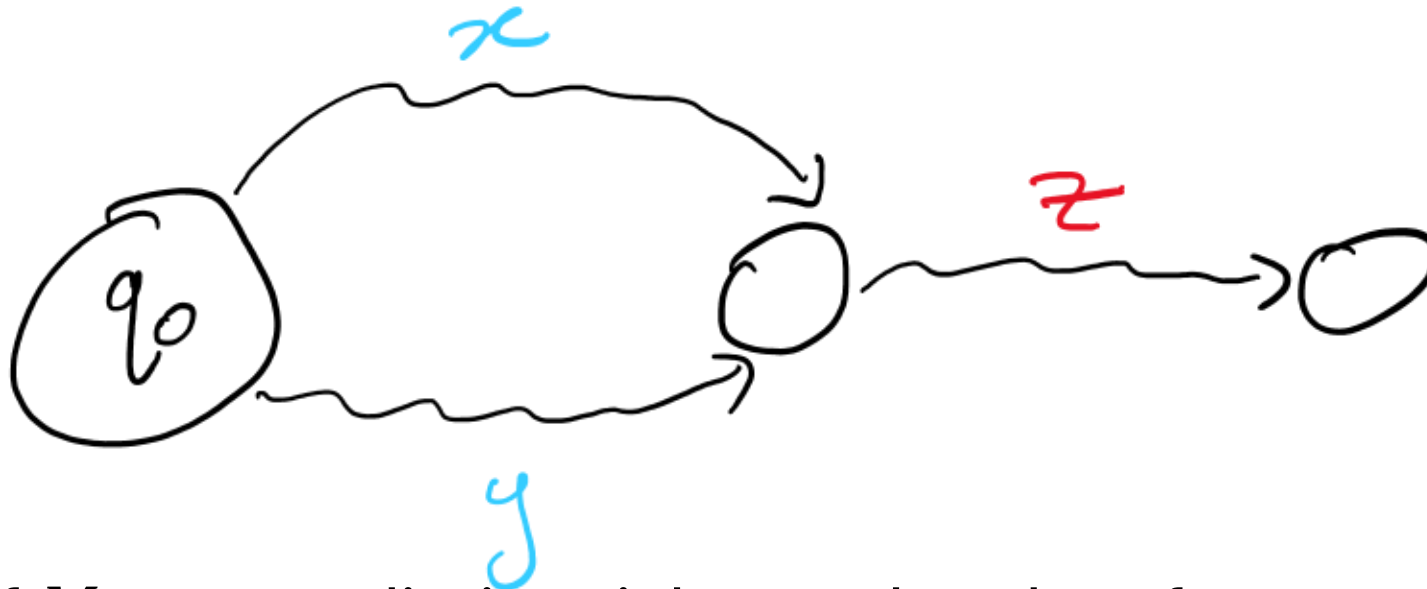


- *Resulting state* of DFA M on input string w = final state after processing $w = q(w)$
- M accepts w if and only if $q(w)$ accepting
- **Key Idea:** If $q(w) = q(u)$, then M cannot distinguish between them and either accepts both or rejects both
- Lemma. A 1-state DFA either accepts all strings or none

Recap: Memorylessness Lemma

If $w = xz$ and $u = yz$ are strings with same suffix z and $q(x) = q(y)$, then $q(xz) = q(yz)$.

Consequently, M accepts both w and u , or rejects both of them



Intuitively, if M cannot distinguish x and y , then for every z , it cannot distinguish xz and yz

Recap: Distinguishable Pairs

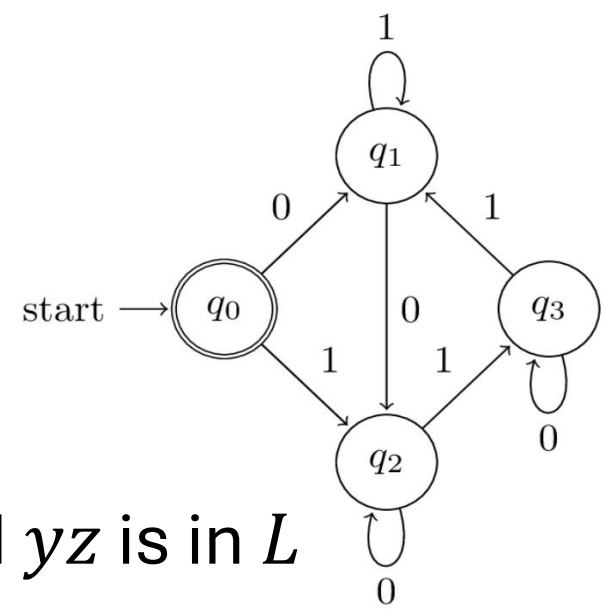
- Let L be a language and x, y, z be strings
- z is a *distinguishing suffix* of x, y if **exactly one** of xz and yz is in L
- x and y is *distinguishable pair* if it has a distinguishing suffix

Example 1

- L = set of even-length bit strings = $\{\epsilon, 00, 01, 10, 11, 0000, \dots\}$
- ϵ and 0 has distinguishing suffix 0 :
 - $\epsilon 0 = 0 \notin L$
 - $00 \in L$

Recap: Distinguishable Pairs

- Let L be a language and x, y, z be strings
- z is a *distinguishing suffix* of x, y if **exactly one** of xz and yz is in L
- x and y is *distinguishable pair* if it has a distinguishing suffix



Lemma. If x, y is distinguishable and M is a DFA such that $q(x) = q(y)$, then M does not recognize L .

Intuitively, M cannot distinguish x, y and so cannot distinguish xz, yz for every z .

Recap: Fooling Sets

A set of strings F is a *fooling set* for language L if:

EVERY pair of strings $x, y \in F$ is distinguishable

Example:

- $L =$ set of bit strings ending in 10 = $\{10, 010, 110, \dots\}$
- $F = \{\epsilon, 1, 10\}$ is a fooling set
- ϵ and 1 has distinguishing suffix 0
- ϵ and 10 has distinguishing suffix ϵ
- 10 and 0 has distinguishing suffix ϵ

Recap: Fooling Set Lemma

If language L has a fooling set F with k strings, then it cannot be recognized by a DFA with less than k states.

Intuitively, a DFA with less than k states does not have enough states (i.e. memory) to distinguish between every pair of strings in F

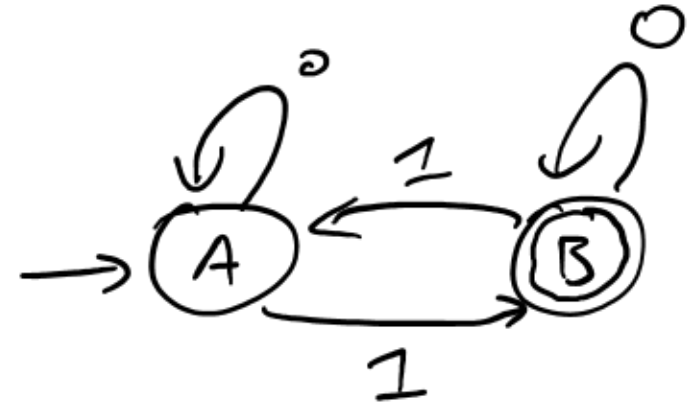
If language L has a fooling set F_k with at least k strings for every positive integer k , then it cannot be recognized by any DFA.

Part 1 Proof in Action

Proof gives algorithm that finds mistake string

Example:

- L = set of bit strings ending in 10 = $\{10, 010, 110, \dots\}$
- $F = \{\epsilon, 1, 10\}$ is a fooling set
- We now use F to rule out following 2-state DFA
- Step 1: Write out resulting states of strings in F
- Step 2: Find 2 strings x, y with same resulting state
- Step 3: DFA makes mistake on xz, yz where z is distinguishing suffix



$$q(\epsilon) = A$$

$$q(1) = B$$

$$q(10) = B$$

1, 10 has distinguishing suffix ϵ

M accepts 1, 10
but $1 \notin L$

Strategies for Constructing Fooling Sets

- Tip 1: Think about information an algorithm that recognizes language needs to keep track
- L = set of bit strings ending in 10 = $\{10, 010, 110, \dots\}$
- $F = \{\epsilon, 1, 10\}$ is a fooling set
- Need to keep track of suffix:
 - Is last symbol 1?
 - Is second last symbol 1 and last symbol 0?
- ϵ is smallest string in which last symbol is not 1
- 1 is smallest string in which 1 is last symbol
- 10 is smallest string in which 1 is second last and 0 is last symbol

Fooling Set Lemma: Application 2

- $L = \{0^n 1^n \mid n \geq 0\} = \{\epsilon, 01, 0011, 000111, \dots\}$
- $F_3 = \{1, 01, 001\}$ is a fooling set
- For every k , the set

$$F_k = \{0^i 1 \mid 0 \leq i \leq k - 1\} = \{1, 01, 001, \dots, 0^{k-1} 1\}$$

is a fooling set.

Proof.

For every pair of strings $x, y \in F_k$,

- $x = 0^i 1$ and $y = 0^j 1$ for $0 \leq j < i \leq k - 1$
- $z = 1^{i-1}$ is a distinguishing suffix
 - $xz = 0^i 1^i \in L$
 - $yz = 0^j 1^i \notin L$ since $j \neq i$

Strategies for Constructing Fooling Sets

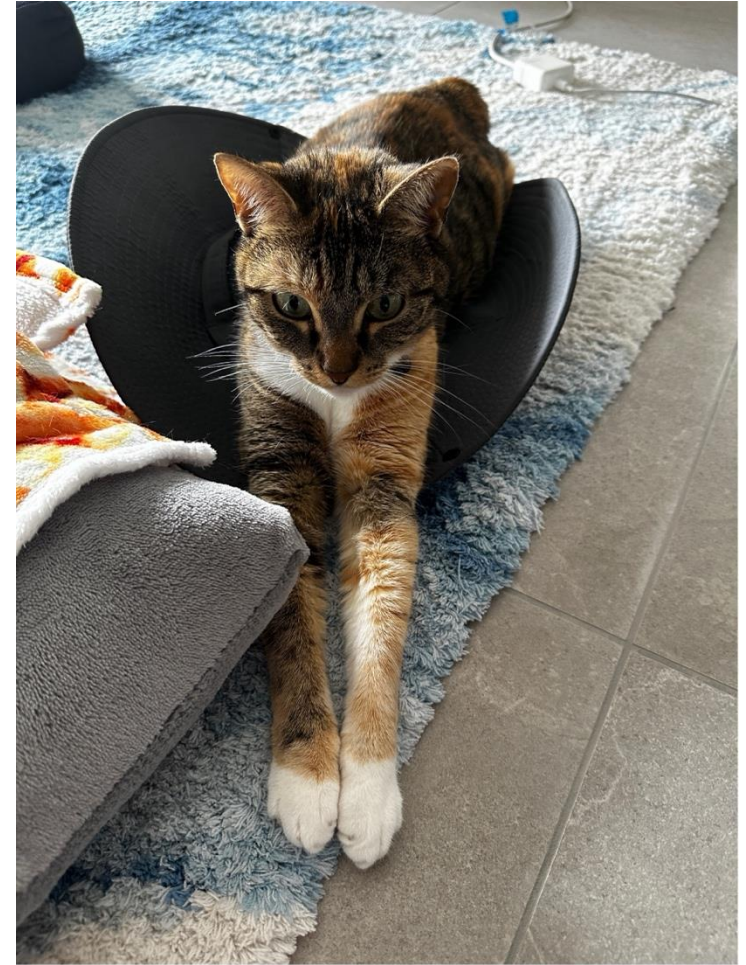
- Tip 1: Think about information an algorithm that recognizes language needs to keep track
- Often easier to start with fooling sets of size 3
- $L = \{0^n 1^n \mid n \geq 0\} = \{\epsilon, 01, 0011, 000111, \dots\}$
- $F_3 = \{1, 01, 001\}$ is a fooling set
- Need to keep track of how many 0s before first 1
- 1 is smallest string in which no 0s before first 1
- 01 is smallest string in which one 0 before first 1
- 001 is smallest string in which 1 two 0s before first 1

Strategies for Constructing Fooling Sets

- Tip 1: Think about information an algorithm that recognizes language needs to keep track
- Tip 2: Start with fooling set of size 3
- Tip 3: Focus on structured strings

Example: Palindromes

- Let $\text{rev}(w)$ be the reverse of w
- w is a *palindrome* if $w = \text{rev}(w)$
- Example:
 - radar
 - kayak
 - tacocat
- $L = \{w \mid w = \text{rev}(w)\}$ is not regular
- Focus on odd-length strings with exactly one b:
 - b, aba, aabaa are palindromes
 - baa, aab, abaab are not palindromes



Example: Palindromes

- $L = \{w \mid w = \text{rev}(w)\}$ is not regular
- Focus on odd-length strings with exactly one b:
 - b, aba, aabaa are palindromes
 - baa, aab, abaab are not palindromes
- Suppose we only need to be correct on odd-length strings with exactly one b, what information do we need to keep track?
- Ans: How many a's before the b
- $F_3 = \{b, ab, aab\}$ is a fooling set
- Each of these has different #a's before the b
- See notes for more strategies

We don't care if the algorithm is wrong on strings that are not odd-length strings with exactly one b

Proving Nonregularity via Closure Properties

Intersection

- If A and B are regular, then $A \cap B$ is regular
- Implies: If A is regular and $A \cap B$ not regular, then B is not regular
- L = strings with equal number of 0s and 1s
- $\{0^n 1^n \mid n \geq 0\}$ is intersection of L with $0^* 1^*$
not reg *reg* $\Rightarrow L$ not regular
- $L = \{0^n 1^n 2^m \mid n \geq 0, m \geq 0\}$
- $\{0^n 1^n \mid n \geq 0\}$ is intersection of L with $0^* 1^*$

Proving Nonregularity via Closure Properties

Intersection

- If A and B are regular, then $A \cap B$ is regular
- Implies: If A is regular and $A \cap B$ not regular, then B is not regular

- L = even-length palindromes
- L is intersection of palindromes with language of even-length strings
- Does this imply L is not regular?

Proving Nonregularity via Closure Properties

Complement

- If A is regular, then A^c is regular.
- Implies: If A^c is not regular, then A is not regular.
- L consists of strings that are not palindromes
- L is the complement of $\{w \mid w = \text{rev}(w)\}$

Summary

- Algorithm to find mistake string
- Strategies to construct fooling sets
- Proving nonregularity via closure properties: intersection and complement